

vuln-bank

Date: 2025-10-15

Repository <https://github.com/Commando-X/vuln-bank> and a local Docker deployment at <http://localhost:5000>.

Nice — here's the whole inspection rewritten **step-wise**, clear and actionable. Follow each step in your lab to reproduce the inspection, understand the risk, and fix/verify issues.

1. Prepare the lab

- Clone and start the app (Docker recommended):

```
git clone https://github.com/Commando-X/vuln-bank.git
```

```
cd vuln-bank
```

```
docker-compose up --build
```

Open the app at <http://localhost:5000>. Run only in an isolated environment — do not expose to the internet.

```
zsh: corrupt history file /home/kali/.zsh_history
[kali@kali] ~
$ git clone https://github.com/Commando-X/vuln-bank.git
Cloning into 'vuln-bank' ...
remote: Enumerating objects: 272, done.
remote: Counting objects: 100% (128/128), done.
remote: Compressing objects: 100% (62/62), done.
remote: Total 272 (delta 96), reused 78 (delta 77), pack-reused 152 (from 1)
Receiving objects: 100% (272/272), 1.68 MiB | 6.65 MiB/s, done.
Resolving deltas: 100% (137/137), done.

[kali@kali] ~
$ cd vuln-bank

[kali@kali] ~/vuln-bank
$ docker-compose up --build
[+] Running 15/15
  db Pulled
    ✔ c6716127167 Already exists
    ✔ 101e1eb3331 Pull complete
    ✔ e0d0eab899f Pull complete
    ✔ f9070120992 Pull complete
    ✔ d6d67b088a8 Pull complete
    ✔ 293b16f6a2e Pull complete
    ✔ 751839abaa5 Pull complete
    ✔ f3af753089a Pull complete
    ✔ 80ff5a1ba6c Pull complete
    ✔ d1180f35765 Pull complete
    ✔ db742937249 Pull complete
    ✔ 2be8a980278 Pull complete
    ✔ c23affc0ba1f Pull complete
    ✔ 2708180ab07 Pull complete
[+] Building 38.6s (13/13) FINISHED
=> [web internal] Load build definition from Dockerfile
=> [web internal] Load metadata for docker.io/library/python:3.9-slim
=> [web internal] Load dockerignore
=> [web internal] Load context: 28
=> [web 1/1] RUN docker.io/library/python:3.9-slimsha256:5af306dcfbd365788e4d597cb782c5fe0b0238aca384d0f981f466113018
=> resolve docker.io/library/python:3.9-slimsha256:5af306dcfbd365788e4d597cb782c5fe0b0238aca384d0f981f466113018
=> sha256:45963223810aa7507bc950a87f4cf901dc45f91cd508030e374436790 / 5.10MB
=> sha256:f87fac28a3288ae10651ae792e0b0c759618978a3f16a79305589c46 / 4.20MB / 4.20MB
=> sha256:1f467af3d525e00a1a754ca778238108a26550bc24f0fa373891a6c930aa9 / 13.80MB / 13.80MB
=> sha256:5af306dcfbd365788e4d597cb782c5fe0b0238aca384d0f981f466113018 / 10.36MB / 10.36MB
=> sha256:68121abae161adcc3a7b67402179d31d2097865e0dc32e04a46f02519472b / 1.74KB / 1.74KB
=> sha256:5008450034ee012e30807d0dc03c006308c04bc0251593cc02af95064a04 / 251B / 251B
=> extracting sha256:1f467af3d525e00a1a754ca778238108a26550bc24f0fa373891a6c930aa9
=> extracting sha256:5008450034ee012e30807d0dc03c006308c04bc0251593cc02af95064a04
=> [web internal] Load build context
=> transferring context: 3.70MB
=> [web 1/1] RUN apt-get update && apt-get install -y postgresql-client && rm -rf /var/lib/apt/lists/*
=> [web 1/1] WORKDIR /app
=> [web 1/1] COPY requirements.txt .
=> [web 1/1] RUN pip install --no-cache-dir -r requirements.txt
=> [web 1/1] RUN mkdir -p static/uploads templates
```

```
db-1 | 2025-10-18 17:00:47.568 UTC [48] LOG: received fast shutdown request
db-1 | waiting for server to shut down...2025-10-18 17:00:47.574 UTC [48] LOG: aborting any active transactions
db-1 | 2025-10-18 17:00:47.577 UTC [48] LOG: background worker "logical replication launcher" (PID 55) exited with exit code 1
db-1 | 2025-10-18 17:00:47.577 UTC [50] LOG: shutting down
db-1 | 2025-10-18 17:00:47.601 UTC [48] LOG: database system is shut down
db-1 | done
db-1 | server stopped
db-1 |
db-1 | PostgreSQL init process complete; ready for start up.
db-1 |
db-1 | 2025-10-18 17:00:47.724 UTC [1] LOG: starting PostgreSQL 13.22 (Debian 13.22-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2025-10-18 17:00:47.724 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2025-10-18 17:00:47.724 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2025-10-18 17:00:47.721 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2025-10-18 17:00:47.739 UTC [63] LOG: database system was shut down at 2025-10-18 17:00:07 UTC
db-1 | 2025-10-18 17:00:47.770 UTC [1] LOG: database system is ready to accept connections
web-1 | Failed to connect to database (attempt 1/5): connection to server at "db" (172.19.0.2), port 5432 failed: Connection refused
web-1 | Is the server running on that host and accepting TCP/IP connections?
web-1 |
web-1 | Retrying in 2 seconds...
web-1 | Database connection pool created successfully
web-1 | Database initialized successfully
web-1 | * Serving Flask app "app" (lazy loading)
web-1 | * Environment: production
web-1 | WARNING: This is a development server. Do not use it in a production deployment.
web-1 | Use a production WSGI server instead.
web-1 | * Debug mode: on
web-1 | * Running on all addresses.
web-1 | WARNING: This is a development server. Do not use it in a production deployment.
web-1 | * Running on http://172.19.0.3:5000/ (Press CTRL-C to quit)
web-1 | * Restarting with stat
web-1 | * Debugger is active
web-1 | * Debugger PIN: 386-344-121
```

2. Quick repository scan

- Open key files: `app.py`, `auth.py`, `database.py`, `docker-compose.yml`, `.env.example`.
- Read `README` to learn which vulnerabilities are intentionally included.

3. Inspect authentication & sessions

- Look for raw SQL in login/auth code and where JWTs or tokens are created/stored.
- Check if tokens are placed in `localStorage` or cookies and whether they expire or can be revoked.

4. Check for SQL Injection (conceptual test)

- In the login form, test a basic SQLi payload (lab-only, conceptual):
 - Submit username: `' OR '1'='1` and any password.
- If login succeeds, SQLi exists. Note the file and line where the DB query is built.

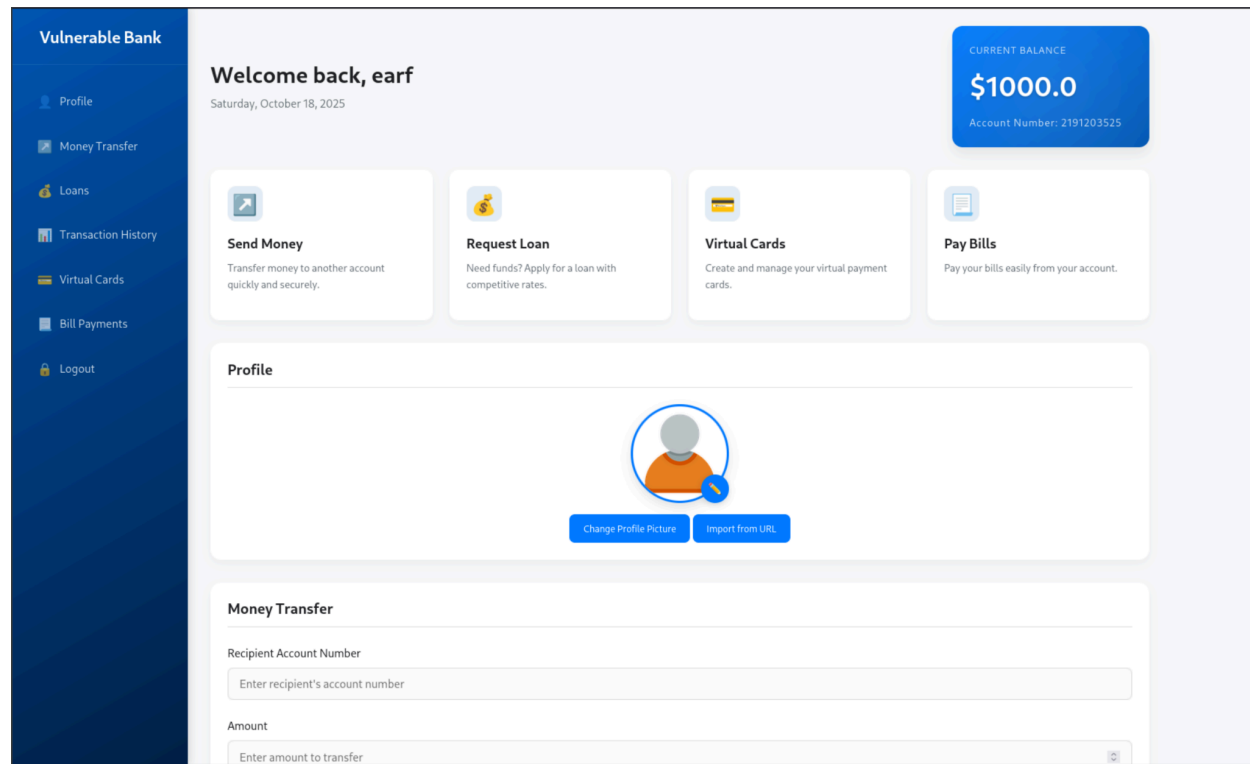
5. Verify object authorization (IDOR / BOLA)

- Log in as user A.
Change resource IDs in the browser/API (e.g., `/api/users/2/transactions`) to a different user ID.
- If you can access another user's data, record the endpoint and missing authorization check.

6. Test file upload and URL import behavior

- Try uploading a file with an unusual extension or filename like `../../../../etc/passwd`.

- Try using any “import by URL” feature pointing to <http://127.0.0.1:3000> or other internal addresses.
- If uploads are served from webroot or internal URLs are fetched, flag SSRF / path traversal / unsafe upload handling.



7. Review transaction logic and concurrency

- Attempt transfers with:
 - Negative amounts.
 - Zero amounts.
 - Two simultaneous transfers that withdraw the same funds.
- If any of these succeed or balances become inconsistent, note missing server-side validation and lack of DB atomicity/locking.

8. Search for XSS and CSRF

- Find form fields or profile fields that are displayed back to users (comments, profile name).
- Try injecting a simple script payload (`<script>alert(1)</script>`) in a lab input and see if it executes.
- Check whether state-changing POSTs have CSRF tokens.

9. Inspect LLM/AI helper usage

- Locate where prompts are constructed for the AI agent.
- Verify that secrets (env variables, DB content) are not included in prompts.
- Try a prompt-injection-style input (lab-only) to see if the model echoes internal info.

10. Prioritize and document findings

- Create a short list of findings with severity:
 - High: SQLi in auth, broken sessions/JWT, IDOR, unsafe file imports (RCE/SSRF), transaction flaws.
 - Medium: XSS, CSRF, LLM prompt leakage.
- For each finding record: endpoint/file, brief PoC steps, and impact statement (one sentence).

11. Apply immediate mitigations (actions to do first)

- Remove `.env` from repo; rotate any real secrets.
- Run only in a private lab (block from public networks).
- Add basic input validation and remove debug output in runtime.

12. Dev fixes (concrete priorities)

- Replace raw SQL with parameterized queries or ORM prepared statements.
- Enforce ownership checks on each resource access (server-side).
- Harden auth: httpOnly secure cookies or short-lived JWTs + token revocation.
- Validate and sanitize file uploads (MIME, extension, whitelist); store outside webroot.
- Add server-side transaction validation and DB transactions/locking.
- Escape output, add CSP, enable CSRF tokens on forms.
- Sanitize inputs used in LLM prompts and never include secrets.

13. Verify fixes (post-patch tests)

- Re-run the SQLi login payload — it must fail.
- After fixes, try accessing another user's ID — you must get denied (403/401).
- Attempt a suspicious file upload or internal URL fetch — it must be blocked.
- Simulate concurrent transfers and confirm balances remain correct.
- Re-test stored/reflected XSS vectors and confirm payloads are escaped.

14. Conclusion

I gain hands-on experience on various vulnerabilities and their effect on vulnerable sites.

